Latest updates: https://dl.acm.org/doi/10.1145/2512840.2512867

RESEARCH-ARTICLE

Dynamic frequency scaling on android platforms for energy consumption reduction

PAMELA THAYS L BEZERRA, Federal University of Pernambuco, Recife, PE, Brazil

LEANDRO A B ARAUJO, Federal University of Pernambuco, Recife, PE, Brazil

GIOVANE BOAVIAGEM RIBEIRO, Federal University of Pernambuco, Recife, PE, Brazil

ANTONIO CORREIA DE SA BARRETO NETO

ABEL GUILHERMINO DA SILVA-FILHO, Federal University of Pernambuco, Recife, PE, Brazil

CLAUIRTON A SIEBRA, Federal University of Pernambuco, Recife, PE, Brazil

View all

Open Access Support provided by:

Federal University of Pernambuco

University of Pernambuco

PDF Download
2512840.2512867.pdf
30 January 2026
Total Citations: 11
Total Downloads: 381

Published: 03 November 2013

Citation in BibTeX format

MSWiM '13: 16th ACM International Conference on Modeling, Analysis and Simulation of Wireless and Mobile Systems
November 3 - 8, 2013
Barcelona, Spain

Conference Sponsors:
SIGSIM

PM2HW2N '13: Proceedings of the 8th ACM workshop on Performance monitoring and measurement of heterogeneous wireless and wired networks (November 2013)
https://doi.org/10.1145/2512840.2512867
ISBN: 9781450323710

Dynamic Frequency Scaling on Android Platforms for Energy Consumption Reduction

Pamela Bezerra; Leandro Araujo; Giovane Ribeiro; Antonio Neto; Abel Silva-Filho; Clauirton Siebra; Fabio, Q. B. da Silva, Andre Santos
Informatics Center (CIn)
Federal University of Pernambuco
Recife-PE, Brazil
{ptlb, laba, gbr2, acsbn, agsf, cas, fqbs, alms}@cin.ufpe.br

Angelica Mascaro; Paulo Costa
CIn/Samsung Laboratory of Research and Development
Federal University of Pernambuco
Recife-PE, Brazil
{aam3, phrc}@cin.ufpe.br

ABSTRACT

Dynamic Voltage and Frequency Scaling (DVFS) is an efficient energy saving technique for processing units. This paper evaluates the impact of the DVFS on the energy consumption, when it is applied to adjust the operational frequency of an Android based smartphone during common mobile activities, such as 3G and Wi-Fi communications. An experimental infrastructure was defined to carry out energy measurements, using a simulated environment and a Samsung Galaxy SII smartphone. After defining the best operational frequency for each activity, we identified the amount of energy that could be saved if the smartphone was using an optimized strategy to adjust the frequency. Results of the proposed approach were compared to the *performance* mode of this smartphone and an average energy reduction of about 23.4% was obtained.

Categories and Subject Descriptors

C.2.1 [**Network Architecture and Design**]: Wireless Communications.

General Terms

Measurement, Experimentation, Human Factors.

Keywords

Embedded Systems; Energy-aware Design; Android; DVFS; Operating System.

1. INTRODUCTION

Dynamic Voltage and Frequency Scaling (DVFS) is a technique [1] that can be used to reduce the consumption of energy via the adjustment of operational frequency and voltage of devices, in accordance with their status and performance constraints. The use of this method in mobile devices, such as smartphones, tablets and notebooks enables an increase in the autonomy of use, which is maintained by batteries. In fact, this strategy is already used in embedded systems because such systems do not usually require a high operational frequency during all the time.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than ACM must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.
PM²HW²N'13, November 3–8, 2013, Barcelona, Spain.
Copyright © 2013 ACM 978-1-4503-2371-0/13/11…$15.00.

In recent years, the mobile devices market has been evolving rapidly in terms of amount and variety of applications. According to [2], in February of 2012, 49.7% of mobile owners had smartphones and 48% of these smartphone were Android handsets. This increase tends to continue and by 2013, smartphones will probably account for 70% of all US mobile devices. The technological advances and reduction of electronic components are supporting the implementation of more complex functions, following the Moore predictions. In fact, the cellphones, which were only used to voice calls, now support services such as mobile Internet, maps-based services, multimedia applications, data connectivity and others. In addition, several network protocols are available, such as Wi-Fi, Bluetooth, 3G and now 4G.

Unfortunately, the limited battery lifetime has always been the bottleneck when it comes to the development of improved portable electronic products. In addition, constraints in the size and weight of mobile phones prohibit the use of heavy and large battery packs as power sources. Although the battery technology has been improved over the years, it definitely has not kept up with the advances in other technological fields or the energy demands of wireless platforms. This fact has motivated the development of strategies that reduce the energy consumption of these devices without changes in the hardware architecture.

To reduce this gap between battery and complex smartphones, the scientific community has been studying many power saving techniques (DVFS, DPM, MAR-CSE, etc.). These techniques employ energy saving without impact the software performance or carry out great changes in the hardware infrastructure. Among these techniques, the DFVS, which is very common in desktops and servers, works with the frequency and voltage scaling of the processor, increasing the frequency only when there is workload. Higher frequencies bring better performance, but also higher voltages and temperatures to the processor, which causes waste of energy. DVFS, therefore, reduces the waste of energy decreasing the frequency when there is few or no workload.

This work studies the implementation of a simple DVFS technique in Android smartphones. Our DVFS is user-driven and scale the frequency according to the application that is in foreground. When an application is used, the processor is not the unique component working, but many others (like display, memory, GPS, communications interfaces, etc.) are also activated, which means that the simple reduction of the processor frequency would not bring energy saving to the system. Therefore, our study

generates a relation between frequency and application: each application has an optimized frequency, where there is the best trade-off between power saving and performance. For that end, we first carried out an experimental study to characterize the energy profile of several common applications of a device. This paper focused on the 3G and Wi-Fi applications, so that we could evaluate the effects of the energy consumption when we apply the DVFS during operations using such technologies.

This work focuses on devices that use the Android operating system. According to the Gartner Institute [3], Android devices represented 52% of the mobile market in 2011, so that Android tends to be the main operating system for mobile devices in a short term. Our aim is to provide a strategy to reduce the energy consumption, in Android devices, via the adjustment of the CPU frequency and voltage.

The remainder of this work is structured as follows: Section 2 summarizes some related works that use the DVFS technology and the detailed definition of DVFs and CPU governors. Section 3 describes our experimental environment and initial experiments to validate such environment. Section 4 discusses a set of experiments, whose goal was to characterize the Android behavior regarding the energy use. Section 5 introduces our optimization strategy, while Section 6 discusses the results obtained via our approach. Finally, Section 7 concludes this works with the main remarks and future works.

2. THEORETICAL BACKGROUND

2.1 The DVFS and the governors

The DVFS – Dynamic Voltage-Frequency Scaling- is a technique used to manage the CPU performance and energy consumption. The term dynamic refers to the fact that the frequency scaling occurs during the program's execution time. Changing the frequency determines how many instructions can be executed in a period of time. However, frequency and voltage are proportional. Then higher voltages are necessary to operate higher throughputs. There is, therefore, a trade-off between performance and power saving. The correct management of this trade-off results in a better performance regarding energy saving. Initially applied to notebooks and desktops, the DVFS is also becoming a popular solution to smartphones and tablets.

The *CPUfreq*, a kernel module developed for Linux operating systems, implements the DVFS in Linux and provides a set of modular interfaces for frequency scaling management [11]. This subsystem is composed of the following components:

- *CPUfreq module*: this module provides a common interface between the various low level frequency management technologies and the high level software polices for frequency scaling;
- CPU *Drivers*: these drivers implement many technologies for CPU frequency scaling, like Intel®, SpeedStep® Technology, AMD PowerNow! and Intel Pentium;
- *In-Kernel Governors*: The infrastructure of the Cpufreq executes the frequency scaling through software policies called *governors*.

A *governor* is an algorithm that calculates the appropriate frequency for the processor based on the workload and the number of active processors. There are 5 different *governors* since version 2.6.16 of the kernel Linux [11]:

- *Performance*: always maintains the CPU in the highest frequency;
- *PowerSave*: opposite of *performance*, maintaining the CPU in the lowest possible frequency;
- *Userspace*: move the system frequency from the kernel level to the user level, enabling that users manually configure such frequency;
- *Ondemand*: dynamically scale the frequency based on the CPU workload;
- *Conservative*: the conservative governor is a "fork" in the project of the *Ondemand* governor with a slightly different frequency scaling algorithm, which enables the use of intermediary frequencies;

The *Performance* and *Powersave* are static governors. They always maintain the system at the same frequency whatever the workload is. The *Userspace*, however, is a customized dynamic governor once the user can set the frequency according to her/his needs. The *Ondemand* and *Conservative* governors are also dynamic governors, but they change the system frequency automatically according to the CPU usage. The motivation behind doing the dynamic frequency scaling inside the kernel instead of at user space was to reduce instruction overhead. Furthermore, the kernel is the right place to take the frequency decision as it has others information about the system overall and the particular CPU [10].

The design goal with the original *Ondemand* governor was to reduce frequency with few performance degradation and to keep the code simple. It was a simplistic algorithm which dynamically set the frequency to a maximum value whenever the CPU is in use. The *Conservative* governor, much like the *Ondemand*, sets the CPU depending on the current usage. It differs in behavior once it smoothly increases and decreases the CPU speed rather than simply jumping to max speed when there is any load on the CPU. This behavior is more suitable for a battery powered environment.

The *Ondemand* governor is the default governor of the majority part of the Linux based systems, including notebooks and smartphones.

2.2 Related Works

Reducing the processor to the lowest frequency may not reduce the power consumption. According to [4], reducing the frequency processor may increase the consumption of others components like memory or network interfaces. The paper shows that the frequency that produces a power saving without losing the deadlines is called *critical speed*. In [5], the authors propose a DVFS approach that relates the critical speed with the *memory access rate – MAR*, creating a prediction mathematical equation called MAR-CSE (*Memory-Access Rate Critical Speed Equation*). This mathematical model is the base for the development of an algorithm called AD-DVFS that is implemented in an Android operational system platform, version 1.0. This algorithm calculates the current value of MAR and activates the adequate frequency based on the mathematical model. The AD-DVFs algorithm produced an energy reduction of 2.6 to 11% compared with the lowest frequency, however 22.9 to 46.6% faster than this frequency.

Choi *et al* [6] proposed a DVFS technique, called workload decomposition (DVFS-WD), where the CPU workload is divided into two parts: on-chip and off-chip. The task workload decomposition can be statically carried out by the compiler or dynamically using the Performance Monitor Unit, also called PMU. This technique is based on real-time statistical reports, from the PMU hardware counters, based on the XScale Intel processor [7]. The on-chip loading represents the execution cycles of CPU instructions, while the off-chip loading represents the external access memory cycles. Therefore, this work analyzes the impact of frequency scaling both in CPU and its peripherals. In the tests with the BitsyX system using a set of benchmarks, the DVFS-WD algorithm presented a CPU energy reduction of 80% for memory dependent applications and 20% for CPU dependent applications, with a performance degradation of less than 20%.

In [8] a DVFS technique associated with an efficient power consumption strategy is proposed for a family of ARM processors. This method was developed to reduce energy consumption of real time applications. This article implements an inter-task DFVS technique for real time process called *Deterministic Stretch-to-Fit* (DSF). The DSF uses the *Dynamic Slack Reclamation* (DSR) algorithm proposed in [9], which gives the *Dynamic Slack* of a previous task for the next finished task of higher priority. To verify the results, the DSF was implemented in an ARM1176jzfs real processor and in a simulated Cortex_A9 processor. The authors also simulated the ARM11 processor for comparing results of both simulated and real platform. The software used for simulating the processor was QEMU. The ARM1176jzfs has 4 different frequencies: 160, 215, 240 and 265 MHz. The Cortex_A9 only has three possible frequencies: 300, 600 and 1000 MHz . Appling the new technique for both processors, using for different real time tasks, was observed a reduction in the energy consumption of 30%, varying from 4.04 to 17.73% in the real platform of ARM1176jzfs, 4.08 to 17.81% in the simulated ARM11 and 1.56% to 31.25% for the simulated Cortex_A9.

All these previous works use the DVFS for CPU tasks. The approach proposed in [10] is the first one that uses the user feedback to scale the frequency. The authors introduced a User-Driven Frequency Scaling (UDFS). This technique uses direct user feedback to drive online control algorithm that determines the processor frequency. To understand the user profile, the authors conducted a randomized user study of eight users, comparing four processor frequency strategies including dynamic, static low (1.06GHz), static medium frequency (1.33GHz), and static high (1.86GHz). Three different tasks were performed during 4 minutes using these different frequency scaling methods and a feedback of each user experience was recorded: the user can express discomfort at any time by pressing the F11 button on the keyboard. With this feedback was possible to develop two different algorithms: UDFS1, an adaptive algorithm that uses three parameters to increase or decrease the frequency, and UDFS2, that tries to find the lowest frequency at which the user feels comfortable and then stabilize there. Three applications were tested using an IBM Thinkpad T43P with a 2.13 GHz Pentium M-770 CPU and 1GB memory running Microsoft Windows XP Professional: Microsoft PowerPoint, 3D Shockwave animation and FIFA game. On average, the power consumption can be reduced by 24.9%. For the Shockwave animation, for example, the UDFS2 reduced the consumption by 32.2%.

3. INITIAL SETUP

Our aim in this work is to evaluate the impact of the DVFS strategy on the energy consumption of Android devices, when we are using common mobile services. For that end, we needed to setup our evaluation environment and carry out initial experiments to certify this environment. Next subsections discuss these two activities.

3.1 Experimental Environment

The environment used to evaluate the energy consumption is composed of a Smartphone Samsung Galaxy SII, GT-I9100 model, with the 2.3.4 version of the Android operating system, and a DC Agilent E3640A programmable power source, which is connected to the smartphone a special interface (GT-M7500 JTAG). This source acts like a battery, providing a constant voltage of 3.7V to keep the smartphone working, while collects the value of current passing through the device. The source is also connected to a computer, where an external program, running in a host computer, captures information about voltage and current that is provided to the smartphone. This program, the Samsung Energy Profile, was developed by our research group and it is able to continuously store 9 samples per second (voltage and current tuples) in real-time along the experiment. This environment is described in Figure 1.

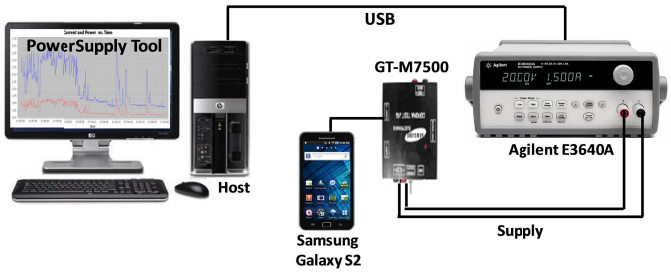

Figure 1: Infrastructure for mobile energy measurements

3.2 DVFS method applied to Android

DVFS is becoming a popular method in the mobile area, such as mobile phones or tablets, once it presents a huge potential for energy saving. In order, a considerable part of the energy consumption is related to the dynamic power, which is directly proportional to the operational frequency and quadratic proportional to the CPU voltage. Using this relation, it is easy to understand that if we use the lowest possible voltage and frequency, we could consequently obtain the lowest power dissipation to a specific application. Note, however, that when we reduce the operational frequency, the CPU becomes slower, carrying out less operations, and this may increase the energy consumption.

Several works only consider the CPU behavior when they propose static models or strategies to reduce the energy consumption. However, in real applications, there are other factors that are away of an ideal execution model. For example, a CPU usually needs to perform stall and wait operations to memory access. Other aspects, such as the use of Wi-Fi, 2G and 3G technologies, may also affect the system efficiency when we have a frequency/voltage modification. This may cause non-conventional behaviors and loss of service quality.

For our experiments we have used the CPUMaster [12], which is a tool that explicitly manages the CPU frequency in an Android Smartphone with root privileges. When a frequency is modified, the voltage is also accordingly changed. The Samsung Galaxy SII

smartphone has a default operational frequency of 1200 MHz. Using the CPUMaster, we can set this frequency to four other different values: 1000 MHz, 800 MHz, 500 MHz and 200 MHz. In addition, it is also possible to configure this device to the five different governors previously described: *Ondemand*, *Conservative*, *Performance, Userspace* and *Powersave*. To measure the effect of frequency on the device and the relation between power consumption and different levels of frequency, we have carried out measurements of current, voltage and power for three simple applications using the five Samsung Galaxy SII possible frequencies. The three applications carried out, respectively, 10,000,000 sum, multiplication and division operations. The results are show in the next graphs (Fig. 2).

Based on these simple experiments, we see that the lowest frequency does not always correspond to the lowest energy consumption (Fig. 2c). This happens because when we decrease the frequency, the execution time of applications tends to increase (Fig. 2b). Considering this fact, when applications mainly depend on the smartphone CPU, the power and execution time behavior is predictable. This means, the frequency increasing causes higher power dissipation (Fig. 2a).

According to the results of these experiments, the CPU operational frequency that causes the lowest energy consumption is 800 MHz to sum applications, 1 GHz for multiplication and 800 MHz for division applications (Fig. 2c). Other types of applications certainly have different optimal operational frequencies, regarding the energy consumption, which could be defined via similar experiments.

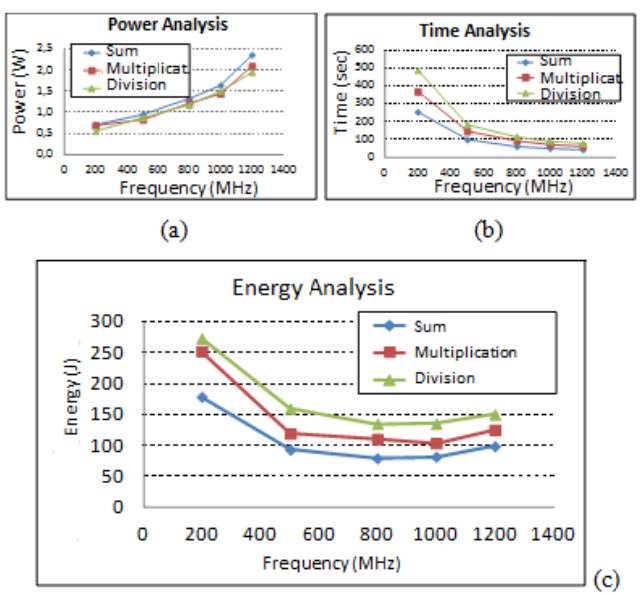

Figure 2: Graphics showing the average power (a), execution time (b) and energy consumption (c) for different applications running in the Samsung Galaxy SII smartphone

4. CHARACTERIZATION OF ANDROID TECHNOLOGY

This section shows the results regarding the characterization of the energy consumption for data communication modes (3G and Wi-Fi), voice call, idle mode and eBook reading considering five different CPU operational frequencies (200 MHz, 500 MHz, 800 MHz, 1000 MHz and 1200 MHz) and two default governors, the *Conservative* and *Ondemand*. For each operational frequency, we have executed 30 experiments.

In total, we have measured the average power of 5 different applications:

1. Idle Mode (no applications);
2. Sending e-mail via 3G(using Gmail);
3. Sending e-mail via Wi-Fi(using Gmail);
4. Making a 1 minute voice call;
5. Reading an eBook(using Saraiva eReader);

Figure 3 summarizes our experimentation process. For each application, we collected current and power of the smartphone in the five different fixed frequencies and in the two dynamic governors. The frequency/governor was configured using CPUmaster at the beginning of the test. Each test case was repeated 30 times. After that we calculated the average power of these 30 executions to determine the power consumption of each frequency/governor.

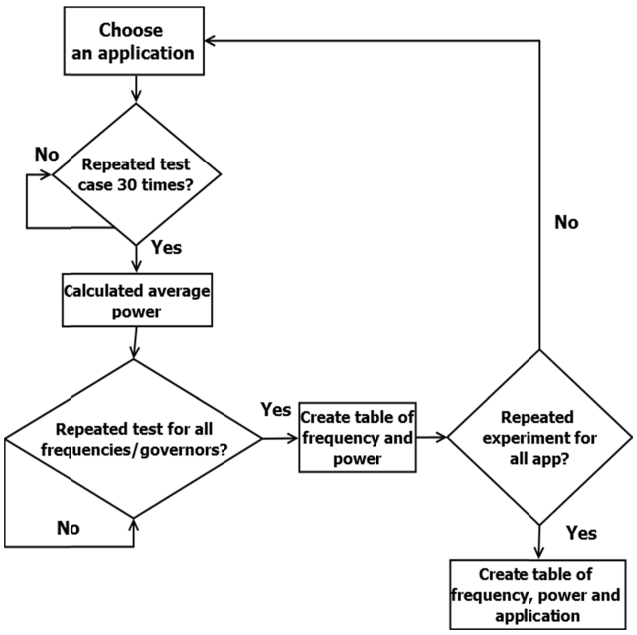

Figure 3: Test flow of the experiments. For each application, we measured the average power of 30 tests, repeating that for all the frequencies.

4.1 Idle Mode

Before the examination of frequency impact on different communication profiles and applications, we have analyzed the variation of frequency in the idle mode. The results can be seen in the next table (Table 1).

A smartphone is in idle mode when no application is active, communications (3G and Wi-Fi) are turned off and users are not interacting to the device. This is the mode where smartphones spend most of their time. The test was performed leaving the smartphone in idle mode for 1 minute. This same behavior was repeated 30 times and an average power was calculated. The variation of energy consumption between the frequencies is very small. The values of the samples were also very closer, with a variance less than 0.019% for most cases. Even though, according to the table, the lowest power dissipation is obtained when the

smartphone is not operating in the lowest frequency, but in the 1000MHz frequency.

Table 1: Power in different frequencies and default governors for idle mode

Idle Mode	
Frequency (MHz)	**Power [mW]**
200	17.02
500	17.16
800	20.44
1000	**16.56**
1200	21.37
Conservative	17.41
Ondemand	18.64

We can also see that the dynamic governors in this case are not very efficient once they consume more power than the lower fixed frequencies, probably because they don't decrease the frequency to the lowest level for performance reasons: they keep expecting new workloads before decrease the frequency.

4.2 3G Data Communications

To evaluate the frequency effects on activities that involve data communication, we have measured the energy consumption and device performance during the sending and reading of emails using both Wi-Fi and 3G technologies. For these measurements, we sent an e-mail message with ten words during an iteration of 3 minutes (the whole iteration corresponds not only to the act of sending the message, but also to open the application, the inbox, write the words, etc). The application used to e-mail reading was the gmail for Android. The same conditions and words were used to execute all measurements.

These tests of communication were made using the real 3G network: the smartphone had a VIVO (a Brazilian communication company) chip and was tested in a room with good 3G signal. Using a real network is possible to observe all usual behavior of a real network, such as handovers (horizontal and vertical), variations in the signal strength or long distance between the communications towers, which makes the energy analysis closer to the real scenario of a person using 3G for navigating.

Table 2: Communication power for 3G using 5 different frequencies and the default governors

3G data communication	
Frequency (MHz)	**Power [mW]**
200	556.89
500	608.58
800	592.05
1000	**553.51**
1200	628.49
Conservative	546.09
Ondemand	602.75

The next table (Table 2) shows the results for measurements using the 3G technology. In this case, the dependency of power and frequency has a great variation, having peaks of energy at 1200MHz and 500MHz, and valleys at 1000MHz and 200MHz. This behavior is caused by the irregularities of the real 3G network already explained. The best frequency for operating the 3G communications is, however, 1000MHz. These tests showed a variation of about 0.4% for the different test cases.

The dynamic governors presented a greater power consumption comparing to 1000, 800 and 200 MHz frequencies, which implies that using a dynamic scaling is no necessarily more efficient in terms of communication than a fixed frequency.

4.3 Wi-Fi Data Communications

The same 5 frequencies, used during the 3G experiments, were considered to carry out the Wi-Fi (IEEE 802.11 network) experiments. The Wi-Fi protocol is a short-distance protocol and, for that reason, it consumes less energy than 3G technology. In fact, the long distance between base station and device is the main aspect for the high 3G energy consumption. In addition, while the use of 3G generally needs several handoffs, the Wi-Fi technology does not present this requirement, once users are always connected to the same base stations during a communication session. Table 3 shows the dissipated power values when we use the Wi-Fi technology. Different of the 3G experiments, we have less variation between the power values and lower power consumption for lower frequencies, but the dynamic governors have similar consumption of the lowest frequency, showing a similar efficiency in the frequency scaling. These experiments showed a variation of about 0.2%.

Table 3: Communication power for Wi-Fi using 5 different frequencies and the default governors

Wi-fi data communications	
Frequency (MHz)	**Power [mW]**
200	**423.37**
500	529.79
800	461.28
1000	694.99
1200	600.76
Conservative	427.25
Ondemand	467.84

4.4 Voice Calls

This scenario used two Samsung Galaxy SII smartphone, model GT-I9100, and voice calls of 1 minute. The experimental process was the same - 30 experiments using the same five frequencies and the two governors. Table 4 shows the results for that scenario.

In voice call we have a different behavior: lower frequencies have higher power consumption. The lowest consumption is when the smartphone is in the highest frequency of 1200MHz. This can be explained by the higher efforts to establish connection in lower frequencies, what makes the communications devices spend more energy than the necessary. Furthermore, the dynamic governors showed high energy consumption, which implies that dynamic scaling for phone calls is not the best choice for reducing power consumption. This can be explained by the fact that voice calls are less dependent on CPU and more dependent on the peripherals devices, such as the smartphone antenna.

Table 4: Communication power for voice calls using 5 different frequencies and the default governors

Voice Calls	
Frequency (MHz)	**Power [mW]**
200	944.20
500	1,013.45
800	850.31
1000	846.49
1200	**833.76**
Conservative	871.80
Ondemand	1,116.49

Also, to perform a voice call requires efforts from both the smartphone and the communication stations antennas, which means it is also dependent of the radio signal and the distance between device and communication company antenna. During the experiments, we have observed a loss in efficiency when we operated in low frequencies, once the system did not promptly response to the input of data via virtual keypad. In addition, the establishment connection time for voice calls spent longer time than the normal.

4.5 eBook Reading

The application used to eBook reading was the *Saraiva Reader*. The experiment was divided into 30 iterations of 2 minutes each. The test consisted of opening a book, reading a page for 30 seconds, pass 10 pages and change the book. This process was repeated to each of the 5 frequencies and the two governors, as well as in the previous cases. The variation of these tests was around 0.9%. The results are summarized in Table 5 below.

Table 5: Power used during the eBook reading in 5 frequencies and two governors

eBook Reading	
Frequency (MHz)	**Power [mW]**
200	996.29
500	1,030.18
800	**977.80**
1000	983.15
1200	1,361.52
Conservative	1,391.75
Ondemand	1,291.57

The 800MHz frequency presented the lowest power consumption. The dynamic governors presented energy consumption similar to the highest frequency. Thus, it is not an efficient approach in this scenario. Reading an eBook is an activity that demands more power from display rather than CPU. The task of changing pages or books is not processing expensive and the user spends great part of the time reading a page. Changes in the frequency are not perceptive to the user and it is possible to work always in the lowest frequency without causing performance loss. That is the reason that dynamic governors are not efficient, once scaling the frequency is not necessary and demands interruptions of the CPUfreq kernel modes. The power consumption of the Conservative governor, for example, was higher than the consumption of the of 1200 MHz frequency

5. OPTIMIZATION STRATEGY

The previous section presented the results for 7 different scenarios, which are common in the use of a smartphone. We can take some conclusions from these experiments. First, regarding Wi-Fi and 3G, the best energy saving frequency is 1000 MHz and 200 MHz, respectively. During the tests, the performance in this low frequency was almost the same of the other frequencies. In the phone calls, however, it was observed a system degradation when it was operating in low frequencies. As the highest frequency is the more power saving, there is no need to worry about the trade-off performance/power consumption in this case. The eBook reading had the less power consumption at the frequency of 800MHz. As it was not observed any performance degradation by the tester (in fact, both in the eBook reading and idle mode was not observed system degradation even in the lowest frequency) this frequency was chosen as the best frequency to this application. All the information about the appropriate frequency, which means, the frequency with best performance-power consumption trade-off, to each activity/technology is summarized in Table 6.

Table 6: Proper frequency for each technology

Mode	Proper Frequency (MHz)	Power [mW]
Idle	1000	16.56
3G	1000	553.51
Wi-Fi	200	423.37
Voice call	1200	833.76
eBook	800	977.80

Based on this frequency characterization, our strategy to optimize the energy consumption intends to adjust the operational frequency according to the activity in use. Then, we just need to detect the activity in use and automatically change to the associated frequency.

The automatic mechanism for detection is in ongoing development and its conclusion is our next goal. In order, the main aim of the current stage of this work is to show the efficiency of the DVFS technique for smartphone, considering an energy saving strategy for different users' profiles. These profiles, for our work, indicate how users employ their devices regarding the technologies discussed in this paper (3G, Wi-Fi, voice call and eBook reading).

6. RESULTS

Figure 4 summarizes the average power for each application at different frequencies and governors. We can observe that for the great part of applications the power consumption is not proportionally linear to the frequency. In phone calls, for example, the highest consumption occurs in the frequency of 500 MHz. Another point is that the dynamic governors are not necessarily efficient in some cases. Comparing these governors with the fixe frequencies, their energy consumption is, in many cases, closer to the higher frequency consumption. Looking the phone calls as an example, we can observe that the consumption of the *Ondemand* governor is greater than the consumption in the 500 MHz

frequency. Comparing to the governors, the *Conservative* presented better energy conservation in many cases, proving that the *Conservative* is an improvement of the *Ondemand* governor, as explained in section 2.1.

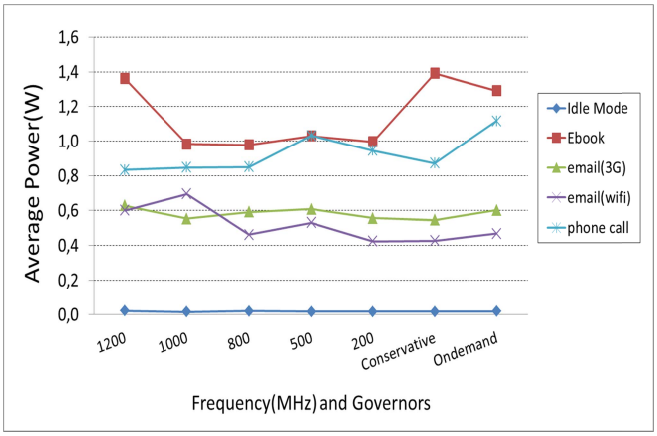

Figure 4: Comparison between the required powers for Idle, 3G, Wi-Fi, voice call and eBook reading in 5 different CPU operational frequencies and the governors.

Using the results about characterization of the energy consumption above, we have manually defined several use profiles for the Samsung smartphone regarding activities of voice call, emails and document reading. The main idea is to understand the energy consumption behavior along a complete day (24 hours), using the five frequencies available in this device and the two dynamic governors, and how the consumption could behave if we apply the optimization strategy where each technology operates in its optimal frequency.

In this context, we have defined some theoretical profiles, considering the use of the technologies that are commonly employed in a smartphone. We defined the duration of use of each of the applications tested during a day and we used the average power of these applications to estimate the total energy consumption in 24h. We made this estimation for the five different static frequencies and the dynamic governors.

The results of total energy consumption, in Joules, along the day are summarized in the next table (Table 7).

Table 7: Summary of results presented in Annex I

Frequency (MHz)	Energy [Joules]
200	11,463.27
500	12,703.29
800	11,741.64
1000	13,187.37
1200	14,606.88
Conservative	13,135.13
Ondemand	13,584.56
Optimized	11,188.51

These results show that the energy consumption, in 24 hours of use, increases in accordance with the increase of frequency values. This phenomenon is almost a linear relation, considering the frequencies between 200 MHz and 1200 MHz. Comparing the dynamic governors we can observe that the *Conservative* governor has better energy efficiency than the *Ondemand* governor, an expected result since we know that the Conservative is an improvement of the *Ondemand*. However, comparing the governors with the fixed frequencies, they are efficient only when compared to the higher frequencies of 1200 and 1000MHz.

When we use our optimized strategy, this means, we adjust the frequency according to the technology in use, we have a significant saving of energy. Note that the value of energy consumption (11,188kJ) is less than any other value of fixed or dynamic frequency profile. Comparing the energy consumption of the optimized approach to the Samsung Galaxy *Performance* profile, which uses the 1200 MHz frequency, we have a reduction of about 23,40%. Comparing to the dynamic governors, we obtained reduction of 17,63% in relation to the *Ondemand* governor and 14,81% comparing to the *Conservative*.

These results show the importance of scaling the frequency not only observing the CPU workload, as does the dynamic governors, but also the application used at the moment. Each Android application has your optimal frequency. That is, a frequency where it normally operates, while dissipates the lowest possible power. The challenge of automatically finding and using this frequency is a very promising approach to create more efficient frequency scaling methods.

7. CONCLUSION AND RESEARCH DIRECTIONS

This paper carried out a study about the energy consumption behavior of a smartphone, considering five different frequencies and two CPU dynamic governors: *Conservative* and *Ondemand*. This study aimed at understands the impact of frequency scaling for some applications at Android operating system. We also analyzed the performance of common resources such as 3G and Wi-Fi communications. Ever using a small number of technologies and applications, with a theoretical profile obtained by interviews with the research team, we obtained a considerable reduction of energy, so that the use of DVFS considering the profile seems to have a huge potential to implement energy saving strategies.

For future work, our priority is to develop a service that collects data of applications in use. With this service we could analyze a real profile use of many users. These data would enable us to construct many different profiles based on the most used applications and then we could test the efficiency of our method for different smartphone users. Our second priority is to develop an Android service that could automatically change the frequency based on the applications executing in foreground. That service would always observe the Android task manager to detect the application running, and then would make a request to the Android kernel to change the frequency.

We also want to expand our study to other applications and technologies. Future works intend to carry out measurements considering the 4G network, Bluetooth, GPS and the use of several technologies at the same times (we tested only one application per time). We also intend to specify an intelligent mechanism to adjust the operational frequency in an autonomous way even for new downloaded applications. Note that there are several technologies and combinations of use that could be used by smartphone users. An intelligent mechanism could adapt the

frequency setting to the best level, rather than use fixe values to specific technologies. Also, we would like to expand our study to some other peripherals, analyzing the memory access rate, for example, to understand the impact of DVFS not only in CPU.

8. ACKNOWLEDGMENTS

The results presented in this article have been developed as part of a collaborative project between Samsung Institute for Development of Informatics (Samsung/SIDI) and the Centre of Informatics at the Federal University of Pernambuco (CIn/UFPE), financed by Samsung Eletrônica da Amazônia Ltda., under the auspices of the Brazilian Federal Law of Informatics no. 8248/91.

9. REFERENCES

[1] W. Liang and P. Lai. "Design and Implementation of a Critical Speed-based DVFS Mechanism for the Android Operating System". In: International Conference on Embedded and Multimedia Computing (EMC), pp. 1-6, 2010.

[2] http://venturebeat.com/2012/03/29/the-magic-moment-smartphones-now-half-of-all-u-s-mobiles/.

[3] Technology Research. Gartner Inc. Available in: http://www.gartner.com. Accessed in: 28/06/2012.

[4] R.Jejurikar and R.K.Gupta, "Dynamic Voltage Scaling for System wide Energy Minimization in Real-Time Embedded Systems", Proceedings of the International Conference on low power electronics and Design, 2004, pp. 78-81.

[5] Wen-Yew Liang and Po-Ting Lai, "Design and Implementation of a critical speed-based DVFS mechanism for the android operating system", 5th International Conference in Embedded and Multimedia Computing, 11-13 Aug. 2010.

[6] K. Choi, R. Soma, and M. Pedram, "Dynamic Voltage and Frequency Scaling based on Workload Decomposition," in Proc. 2004 Int. Symp. Low Power Electronics and Design, Aug. 2004, pp. 174-179.

[7] Intel XScale® Technology Overview, [online]. Available at http://www.intel.com/design/intelxscale/.

[8] Khan,J., Bilavarn,S. and Belleudy,C., "Energy Analysis of a DVFS based power strategy on ARM platforms", 2012 IEEE Faible Tesion Faible Consommantion (FTFC), 2012.

[9] Bhatti, M.K., Belleudy, C. and Auguin, M., "An Inter-task Real-time DVFS Scheme for Multiprocessor Embedded Systems", 2010 Conference on Design and Architectures for Signal and Image Processing (DASIP).

[10] Aridam Mallik, Bin Lin, Gikhan Memik, Peter Dinda and Robert P. Dick, "User-Driven Frequency Scaling", Computer Architecture Letters, vol. 5, issue 2, July-December 2006.

[11] Pallipadi V. e Starikovskiy A., "The Ondemand Governor", 2006 Linux Symposium, Volume Two, pp. 216- 230.

[12] CPUmaster, an application to change the frequency scaling on rooted smartphones: https://play.google.com/store/apps/details?id=com.antutu.CpuMasterFree.